

Week 8 Game Theory Notes

Jacob Wyngaard

Contents

1	Brown–von Neumann–Nash Dynamical System	2
1.1	Game Setup	2
1.2	Expected Payoffs	2
1.3	Excess Payoff Functions	2
1.4	BNN Dynamics	3
1.5	Equivalent ODE Form	3
1.6	Simulation Results	4
1.7	Time Step Considerations	4
1.8	RK4 Scheme	4
1.9	Convergence Metric	5
1.10	Scaling the Payoff Matrices	5
1.11	Activation Function Experiments	5
2	Fictitious Play	6
2.1	Introduction	6
2.2	Example of an Ordinal Potential	6
2.3	Constructing an Ordinal Potential	6
2.4	A Game without an Ordinal Potential	7
2.5	Matching Pennies as a Simple Counterexample	7
2.6	Alternating and Simultaneous Fictitious Play	7
2.7	Degenerate Games	7
2.8	AFP Iterations	8
2.9	Convergence Theorem	8

1 Brown–von Neumann–Nash Dynamical System

1.1 Game Setup

Consider the two-player game with payoff matrices

$$\Pi_1 = \begin{pmatrix} 7 & 2 \\ 3 & 4 \end{pmatrix}, \quad \Pi_2 = \begin{pmatrix} 1 & 6 \\ 8 & 2 \end{pmatrix}.$$

Let Player 1 use the mixed strategy

$$p^1 = (x_1, y_1), \quad x_1 + y_1 = 1,$$

and let Player 2 use

$$p^2 = (x_2, y_2), \quad x_2 + y_2 = 1.$$

1.2 Expected Payoffs

The expected payoff to Player 1 is

$$u_1(p) = 7x_1x_2 + 2x_1y_2 + 3y_1x_2 + 4y_1y_2.$$

The expected payoff to Player 2 is

$$u_2(p) = x_1x_2 + 6x_1y_2 + 8y_1x_2 + 2y_1y_2.$$

1.3 Excess Payoff Functions

The Brown–von Neumann–Nash dynamics are built from the positive part of the difference between a pure strategy payoff and the current mixed payoff. Define

$$\phi(z) = \max\{0, z\}.$$

For Player 1,

$$\phi_{11}(p) = \max\{0, 7x_2 + 2y_2 - u_1(p)\},$$

$$\phi_{12}(p) = \max\{0, 3x_2 + 4y_2 - u_1(p)\}.$$

For Player 2,

$$\phi_{2A}(p) = \max\{0, x_1 + 8y_1 - u_2(p)\},$$

$$\phi_{2B}(p) = \max\{0, 6x_1 + 2y_1 - u_2(p)\}.$$

These terms measure the *excess payoff* of switching to a pure strategy relative to the current mixed-strategy payoff.

1.4 BNN Dynamics

The Brown–von Neumann–Nash dynamics are

$$\dot{p}_{ij} = \phi_{ij}(p) - p_{ij} \sum_k \phi_{ik}(p).$$

The $\phi_{ij}(p)$ shifts probabilistic weight towards the better strategies. The $-p_{ij} \sum_k \phi_{ik}(p)$ ensures the sum of player probabilities remains at 1.

Note that, at Nash equilibrium, $\phi_{ij}(p) = 0$ for all i, j as no unilateral strategy change benefits either player. This ensures that $\dot{p}_{ij} = 0$ for all i, j , making the Nash equilibrium a stable point of this dynamical system.

For the previous 2×2 example, the dynamical system is

Player 1:

$$\begin{aligned}\dot{x}_1 &= \phi_{11}(p) - x_1(\phi_{11}(p) + \phi_{12}(p)), \\ \dot{y}_1 &= \phi_{12}(p) - y_1(\phi_{11}(p) + \phi_{12}(p)).\end{aligned}$$

Player 2:

$$\begin{aligned}\dot{x}_2 &= \phi_{2A}(p) - x_2(\phi_{2A}(p) + \phi_{2B}(p)), \\ \dot{y}_2 &= \phi_{2B}(p) - y_2(\phi_{2A}(p) + \phi_{2B}(p)).\end{aligned}$$

This is a time-homogeneous system of ordinary differential equations.

1.5 Equivalent ODE Form

Using the probability constraints $x_1 + y_1 = 1$ and $x_2 + y_2 = 1$, this system of ODEs can be rewritten as:

Player 1:

$$\begin{aligned}\dot{x}_1 &= y_1\phi_{11}(p) - x_1\phi_{12}(p), \\ \dot{y}_1 &= x_1\phi_{12}(p) - y_1\phi_{11}(p).\end{aligned}$$

Player 2:

$$\begin{aligned}\dot{x}_2 &= y_2\phi_{2A}(p) - x_2\phi_{2B}(p), \\ \dot{y}_2 &= x_2\phi_{2B}(p) - y_2\phi_{2A}(p).\end{aligned}$$

This form is often easier to interpret, since the probabilities corresponding to inferior strategies evolve in a manner similar to exponential decay, with their probabilistic mass shifted to better performing strategies.

1.6 Simulation Results

Using the initial condition

$$x_1 = y_1 = x_2 = y_2 = \frac{1}{2},$$

I ran an RK45 simulation of the system. The trajectories converge to their mixed Nash equilibrium values

$$(x_1, y_1) = \left(\frac{6}{11}, \frac{5}{11} \right), \quad (x_2, y_2) = \left(\frac{1}{3}, \frac{2}{3} \right).$$

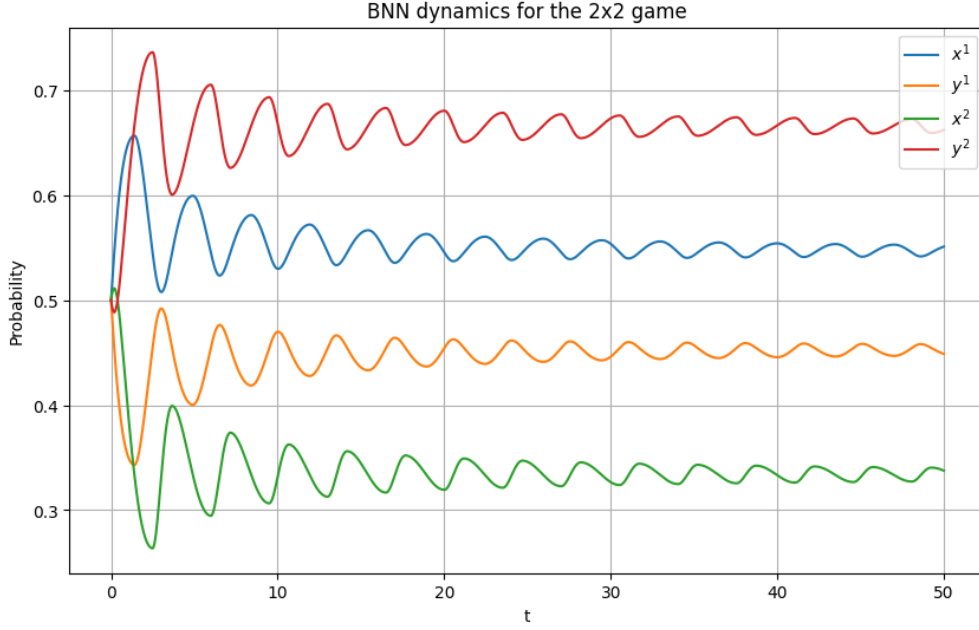


Figure 1: Simulation Results

1.7 Time Step Considerations

I next studied how the choice of time step affected numerical convergence. Since RK45 is an adaptive method that chooses its own step sizes, I switched to the fourth-order Runge–Kutta method (RK4).

1.8 RK4 Scheme

For a general ODE

$$\dot{z} = f(t, z),$$

the RK4 update from t_n to $t_{n+1} = t_n + h$ is

$$\begin{aligned} k_1 &= f(t_n, z_n), \\ k_2 &= f\left(t_n + \frac{h}{2}, z_n + \frac{h}{2}k_1\right), \\ k_3 &= f\left(t_n + \frac{h}{2}, z_n + \frac{h}{2}k_2\right), \end{aligned}$$

$$k_4 = f(t_n + h, z_n + hk_3),$$

$$z_{n+1} = z_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4).$$

In the BNN setting, the vector z is the strategy profile

$$z = (x_1, y_1, x_2, y_2).$$

1.9 Convergence Metric

To measure how close the probabilities at each iteration are to equilibrium, I used the root mean square (RMS) equilibrium error. Defining $p_{i,j}$ as the i th player's j th strategy and $q_{i,j}$ as that probability at Nash equilibrium, define

$$\text{RMS} = \sqrt{\frac{1}{n+m} \sum_{i,j} (p_{i,j} - q_{i,j})^2}.$$

This gives a single scalar measure of distance of the current probability vector from the equilibrium.

1.10 Scaling the Payoff Matrices

I observed that is that, under the ReLU activation function, the BNN system depends linearly on the payoff matrices and is time-homogeneous. Consequently, scaling the payoff matrices by a constant $c > 0$ is equivalent to scaling time.

In other words, if

$$\Pi_1 \mapsto c\Pi_1, \quad \Pi_2 \mapsto c\Pi_2,$$

then the induced BNN flow is essentially sped up by a factor of c .

This explains why the “ideal” fixed time step changes when the payoff matrices are scaled. For the payoff matrices in our example, I found that the best step size was approximately

$$h \approx 1.$$

That said, I will continue to investigate which step sizes ensure convergence of this RK-4 method!

1.11 Activation Function Experiments

I explored replacing the ReLU activation

$$\phi(z) = \max\{0, z\}$$

with smoother alternatives, including:

- tanh,
- sigmoid,
- leaky ReLU,
- GELU,
- ELU.

These smoother activations appeared to be more sensitive to step size and mostly numerically instable. They also may not ensure that the Nash equilibrium is a fixed point of the dynamical system.

2 Fictitious Play

2.1 Introduction

Fictitious play is a discrete-time analogue of Brown–von Neumann–Nash dynamics. It was introduced by Brown and further developed by Robinson, and is often called *Brown–Robinson learning*.

In this framework, the role of excess payoff is replaced by an *ordinal potential* F , which preserves the preference ordering of pure strategies.

Formally, an ordinal potential is a function $F(i, j)$ such that

$$F(i, j) > F(k, j) \iff \Pi_{ij}^1 > \Pi_{kj}^1,$$

and

$$F(i, j) > F(i, k) \iff \Pi_{ij}^2 > \Pi_{ik}^2.$$

F does not preserve payoff magnitudes, only the order of preferences.

2.2 Example of an Ordinal Potential

An example game with ordinal potential corresponds to the following payoff matrices:

$$\Pi_1 = \begin{pmatrix} 5 & 2 & 4 \\ 1 & 4 & 2 \\ 3 & 1 & 3 \end{pmatrix}, \quad \Pi_2 = \begin{pmatrix} 4 & 1 & 3 \\ 2 & 5 & 1 \\ 1 & 2 & 4 \end{pmatrix}.$$

An ordinal potential for this game is

$$F = \begin{pmatrix} 7 & 5 & 6 \\ 1 & 8 & 0 \\ 2 & 3 & 4 \end{pmatrix}.$$

This potential encodes both players' rankings of responses in the case that the other player chooses a pure strategy. From Player 1's perspective, moving vertically changes the payoff ranking; from Player 2's perspective, moving horizontally changes the payoff ranking. The ordinal potential preserves both sets of preferences simultaneously.

2.3 Constructing an Ordinal Potential

For smaller payoff matrices, ordinal potentials can be graphically constructed based on *preference arrows*. The method can be summarized as follows:

1. For each cell of the bimatrix game, draw arrows representing Player 1's preference comparisons along columns and Player 2's preference comparisons along rows.
2. Find a box for which every adjacent preference arrow points outward.
3. Assign this box the next available label, starting at 0.
4. Remove all outgoing arrows from that box.
5. Repeat the procedure iteratively until all boxes are labeled.

For the example above, this construction yields the ordinal potential

$$F = \begin{pmatrix} 7 & 5 & 6 \\ 1 & 8 & 0 \\ 2 & 3 & 4 \end{pmatrix}.$$

This produces a useful interpretation: just as gradients in physics point toward changes in potential, the preference arrows can be used to uncover an underlying discrete potential landscape.

2.4 A Game without an Ordinal Potential

Not all games have an ordinal potential. Consider the following game:

$$\Pi_1 = \begin{pmatrix} 2 & 1 & 3 \\ 3 & 2 & 1 \\ 1 & 3 & 2 \end{pmatrix}, \quad \Pi_2 = \begin{pmatrix} 2 & 3 & 1 \\ 1 & 2 & 3 \\ 3 & 1 & 2 \end{pmatrix}.$$

When we drew the associated preference arrows, there was no cell whose arrows all point outward. While we encountered saddle points, we did not find a minimum of our potential.

2.5 Matching Pennies as a Simple Counterexample

Another standard example is matching pennies, with payoff matrices

$$\Pi_1 = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}, \quad \Pi_2 = \begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix}.$$

This game does not admit an ordinal potential because there is no pure strategy Nash equilibrium. Nondegenerate zero-sum games only have one if they satisfy the minimax = maximin condition with pure strategies. Note that ordinal potentials generally imply sort of collusion benefit.

2.6 Alternating and Simultaneous Fictitious Play

There are two common variants of fictitious play:

- **Alternating fictitious play (AFP):** players update one after the other.
- **Simultaneous fictitious play (SFP):** both players update at the same time.

SFP has been more extensively studied in some settings, but global convergence for nondegenerate ordinal potential games is known only for AFP. That said, global convergence for SFP is known under the stronger assumption of weighted potential structure.

2.7 Degenerate Games

A game is called *degenerate* if at least one of the following holds:

- Multiple Player 1 strategies yield the same payoff against some pure strategy of Player 2.
- Multiple Player 2 strategies yield the same payoff against some pure strategy of Player 1.

Equivalently, degeneracy occurs if there exist indices $i \neq k$ and j such that

$$\Pi_{ij}^1 = \Pi_{kj}^1,$$

or there exist $i, j \neq k$ such that

$$\Pi_{ij}^2 = \Pi_{ik}^2.$$

Nondegeneracy ensures that the preference ordering encoded by the ordinal potential is strict.

2.8 AFP Iterations

Let p_n^1 and p_n^2 denote the mixed strategies after n iterations. These may be initialized arbitrarily, for instance by uniform distributions or by pure strategies.

At iteration $n + 1$, one chooses pure best responses

$$e_{n+1}^1 \in \arg \max_{s_1} s_1^\top \Pi_1 p_n^2,$$

$$e_{n+1}^2 \in \arg \max_{s_2} (p_{n+1}^1)^\top \Pi_2 s_2.$$

The mixed strategies are then updated by empirical averaging:

$$p_{n+1}^i = \frac{1}{n+1} \sum_{k=1}^{n+1} e_k^i = \frac{n}{n+1} p_n^i + \frac{1}{n+1} e_{n+1}^i, \quad i = 1, 2.$$

2.9 Convergence Theorem

Theorem 1 (Berger, 2005). *Let (Π_1, Π_2) be a nondegenerate ordinal potential game. Then every alternating fictitious play process converges to a pure Nash equilibrium.*

The proof is in Appendix A

Appendix A: Proof of Convergence for nondegenerate AFP ordinal potential game

Theorem 2 (Berger, 2005). *Let (Π^1, Π^2) correspond to a nondegenerate ordinal potential game. Then every AFP process converges to a pure Nash equilibrium.*

Proof. We begin by claiming that $F(e_n^1, e_n^2) \leq F(e_{n+1}^1, e_{n+1}^2)$ for all n , where F is the ordinal potential

Since the fictitious play progresses by alternating turns, it suffices to prove the following sub-claims:

1. $F(e_n^1, e_n^2) \leq F(e_{n+1}^1, e_n^2)$ for all n
2. $F(e_{n+1}^1, e_n^2) \leq F(e_{n+1}^1, e_{n+1}^2)$ for all n

Proof of $F(e_n^1, e_n^2) \leq F(e_{n+1}^1, e_n^2)$ for all n

By the selection process of e_n^1 and e_{n+1}^1 , we know that:

- a. $(e_n^1)^T \Pi^1 p_n^2 \leq (e_{n+1}^1)^T \Pi^1 p_n^2$
- b. $(e_{n+1}^1)^T \Pi^1 p_{n-1}^2 \leq (e_n^1)^T \Pi^1 p_{n-1}^2$

By the evolution of p_n^2 , we know that $p_n^2 = \frac{n-1}{n} p_{n-1}^2 + \frac{1}{n} e_n^2$

$$\begin{aligned} (e_n^1)^T \Pi^1 p_n^2 &= \frac{n-1}{n} (e_n^1)^T \Pi^1 p_{n-1}^2 + \frac{1}{n} (e_n^1)^T \Pi^1 e_n^2 \leq (e_{n+1}^1)^T \Pi^1 p_n^2 \\ (e_{n+1}^1)^T \Pi^1 p_n^2 &= \frac{n-1}{n} (e_{n+1}^1)^T \Pi^1 p_{n-1}^2 + \frac{1}{n} (e_{n+1}^1)^T \Pi^1 e_n^2 \leq \frac{n-1}{n} (e_n^1)^T \Pi^1 p_{n-1}^2 + \frac{1}{n} (e_{n+1}^1)^T \Pi^1 e_n^2 \\ \frac{n-1}{n} (e_n^1)^T \Pi^1 p_{n-1}^2 + \frac{1}{n} (e_n^1)^T \Pi^1 e_n^2 &\leq \frac{n-1}{n} (e_n^1)^T \Pi^1 p_{n-1}^2 + \frac{1}{n} (e_{n+1}^1)^T \Pi^1 e_n^2 \\ \implies (e_n^1)^T \Pi^1 e_n^2 &\leq (e_{n+1}^1)^T \Pi^1 e_n^2 \implies F(e_n^1, e_n^2) \leq F(e_{n+1}^1, e_n^2) \end{aligned}$$

Proof of $F(e_{n+1}^1, e_n^2) \leq F(e_{n+1}^1, e_{n+1}^2)$ for all n

By the selection process of e_n^2 and e_{n+1}^2 , we know that:

- a. $(p_{n+1}^1)^T \Pi^2 e_n^2 \leq (p_{n+1}^1)^T \Pi^2 e_{n+1}^2$
- b. $(p_n^1)^T \Pi^2 e_{n+1}^2 \leq (p_n^1)^T \Pi^2 e_n^2$

By the evolution of p_{n+1}^1 , we know that $p_{n+1}^1 = \frac{n}{n+1} p_n^1 + \frac{1}{n+1} e_{n+1}^1$

$$\begin{aligned} (p_{n+1}^1)^T \Pi^2 e_n^2 &= \frac{n}{n+1} (p_n^1)^T \Pi^2 e_n^2 + \frac{1}{n+1} (e_{n+1}^1)^T \Pi^1 e_n^2 \leq (p_{n+1}^1)^T \Pi^2 e_{n+1}^2 \\ (p_{n+1}^1)^T \Pi^2 e_{n+1}^2 &= \frac{n}{n+1} (p_n^1)^T \Pi^1 e_{n+1}^2 + \frac{1}{n+1} (e_{n+1}^1)^T \Pi^1 e_{n+1}^2 \leq \frac{n}{n+1} (p_n^1)^T \Pi^1 e_n^2 + \frac{1}{n+1} (e_{n+1}^1)^T \Pi^1 e_{n+1}^2 \\ \frac{n}{n+1} (p_n^1)^T \Pi^2 e_n^2 + \frac{1}{n+1} (e_{n+1}^1)^T \Pi^1 e_n^2 &\leq \frac{n}{n+1} (p_n^1)^T \Pi^1 e_n^2 + \frac{1}{n+1} (e_{n+1}^1)^T \Pi^1 e_{n+1}^2 \end{aligned}$$

$$\implies (e_{n+1}^1)^T \Pi^1 e_n^2 \leq (e_{n+1}^1)^T \Pi^1 e_{n+1}^2 \implies F(e_{n+1}^1, e_n^2) \leq F(e_{n+1}^1, e_{n+1}^2)$$

Conclusion of Proof

Therefore, since $F(e_n^1, e_n^2) \leq F(e_{n+1}^1, e_n^2)$ and $F(e_{n+1}^1, e_n^2) \leq F(e_{n+1}^1, e_{n+1}^2)$ for all n , $F(e_n^1, e_n^2) \leq F(e_{n+1}^1, e_{n+1}^2)$ for all n

$F(e_n^1, e_n^2)$ is thus a monotonic sequence of n . Furthermore, $F(e_n^1, e_n^2)$ is bounded since the domain of possible (e_n^1, e_n^2) is finite.

By the Monotonic Convergence Theorem, $F(e_n^1, e_n^2)$ must converge to some value V

Let (i, j) be the first pair of strategies (e_N^1, e_N^2) such that $F(e_N^1, e_N^2) = V$.

At their next iteration, Player 1 picks a strategy e_{N+1}^1 such that $F(e_{N+1}^1, j) \geq V$. By monotonic convergence to V , $F(e_{N+1}^1, j) \leq V$. Therefore, Player 1 picks a strategy e_{N+1}^1 such that $F(e_{N+1}^1, j) = V$. By non-degeneracy, i is the only Player 1 strategy e_{N+1}^1 such that $F(e_{N+1}^1, j) = V$. So $e_{N+1}^1 = i$.

At Player 2's next iteration, they pick a strategy e_{N+1}^2 such that $F(i, e_{N+1}^2) \geq V$. By monotonic convergence to V , $F(i, e_{N+1}^2) \leq V$. Therefore, Player 2 picks a strategy e_{N+1}^2 such that $F(i, e_{N+1}^2) = V$. By non-degeneracy, j is the only Player 2 strategy e_{N+1}^2 such that $F(i, e_{N+1}^2) = V$. So $e_{N+1}^2 = j$.

This continues by induction for all $n \geq N$ values, so $p_n^1 \rightarrow i$ and $p_n^2 \rightarrow j$.

Since i is Player 1's best response to p_n^2 for $n \geq N$ and $p_n^2 \rightarrow j$, i is Player 1's best response to Player 2's strategy j by non-degeneracy.

Since i is Player 2's best response to p_n^1 for $n \geq N$ and $p_n^1 \rightarrow i$, j is Player 2's best response to Player 1's strategy i by non-degeneracy.

(i, j) is a pure strategy Nash Equilibrium

□

Appendix B: BNN Solver for Generic $m \times n$ Payoff Matrices

```
1  # %% BNN ODE Solver
2
3  import numpy as np
4  import matplotlib.pyplot as plt
5  import torch
6  import torch.nn.functional as F
7
8  def bnn_ode_solver(p1,p2,A,B,T,N):
9      """
10         Initial Probability Arrays:
11             p1 - Initial Player 1 Probability Array
12             p2 - Initial Player 2 Probability Array
13
14         Payoff Matrices:
15             A - Player 1 Payoff Matrix
16             B - Player 2 Payoff Matrix
17
18         T - Time Period
19         N - Number of Time Steps
20         """
21     m = np.size(A,0)
22     n = np.size(A,1)
23
24     def ode_system(p):
25         """
26             t - Current Time
27             p - Concatenated Probability Array
28             """
29         p1 = p[range(m)]
30         p2 = p[range(m,n+m)]
31
32         # Expected Value Under Current Mixed Strategies
33         current_value1 = p1.T @ A @ p2
34         current_value2 = p1.T @ B @ p2
35
36         # Pure Strategy Payoffs with Opponent's Current Mix
37         pure_payoffs1 = A @ p2
38         pure_payoffs2 = p1.T @ B
39
40         # Phi Function
41         # ReLU Activation Function
42         value_difference1 = torch.from_numpy(pure_payoffs1-current_value1)
43         value_difference2 = torch.from_numpy(pure_payoffs2-current_value2)
44
45         phi_tensor1 = torch.relu(value_difference1)
46         phi_tensor2 = torch.relu(value_difference2)
47
48         phi_vector1 = phi_tensor1.numpy()
49         phi_vector2 = phi_tensor2.numpy()
50
51         # BNN Dynamics
```

```

52     dp1_dt = phi_vector1 - p1*sum(phi_vector1)
53     dp2_dt = phi_vector2 - p2*sum(phi_vector2)
54
55     return np.hstack((dp1_dt,dp2_dt))
56
57
58     # Runge-Kutta 4 Solver
59     step_size = T / (N-1)
60     t_mesh = np.linspace(0,T,N)
61     p_values = np.zeros((m+n,N))
62     p_values[:,0] = np.hstack((p1,p2))
63
64     for index in range(N-1):
65         p_vector = p_values[:,index]
66
67         k1 = ode_system(p_vector)
68         k2 = ode_system(p_vector+step_size*k1/2)
69         k3 = ode_system(p_vector+step_size*k2/2)
70         k4 = ode_system(p_vector+step_size*k3)
71
72         # Stay Within Probabilistic Bounds
73         new_value = p_vector + step_size*(k1+2*k2+2*k3+k4)/6
74         new_vector = np.maximum(new_value,0)
75         new_vector = np.minimum(new_vector,1)
76
77         p_values[:,index+1] = new_vector
78
79     return t_mesh,p_values

```